

End-to-End Data Management Pipeline for a Recommendation System

RecoMart — E-Commerce Recommendation Engine

Data Management for Machine Learning (AIMLCZG529 / DSECLZG529)

Semester: S2-25

Assignment I — EC1 (20 Marks)

Group Number: 43

Programme: M.Tech AI/ML — BITS Pilani WILP

Submission Date: 19.07.2026

Team Member Details

Name	BITS ID	Email
BRAJESH MISHRA	2025AB05161	2025ab05161@wilp.bits-pilani.ac.in
YENGANTIWAR PRASHANT SAINATH	2025AB05178	2025ab05178@wilp.bits-pilani.ac.in
ARTHIKA G	2025AB05180	2025ab05180@wilp.bits-pilani.ac.in
A SRIKAR	2025AB05185	2025ab05185@wilp.bits-pilani.ac.in
GIRIDHARAN B	2025AB05188	2025ab05188@wilp.bits-pilani.ac.in

Table of Contents

1. Problem Statement
2. Objectives
3. Data Sources
4. Methodology / Pipeline Architecture
5. Implementation Details
 - 5.1 Data Generation & Ingestion
 - 5.2 Raw Data Storage
 - 5.3 Data Validation & Quality Report
 - 5.4 Data Preparation & EDA
 - 5.5 Feature Engineering & Transformation
 - 5.6 Feature Store
 - 5.7 Data Versioning & Lineage
 - 5.8 Model Training & Evaluation
 - 5.9 Pipeline Orchestration
6. Results and Output Screenshots
7. Conclusion and Future Scope
8. Google Drive Links

1. Problem Statement

RecoMart is an e-commerce startup that wants to build a data-driven product recommendation engine to improve customer engagement and cross-selling opportunities. The platform collects user behaviour data from multiple channels — web/mobile clickstream logs, transactional purchase history, product catalog metadata, and external API data.

The goal of this project is to design and implement a complete, automated, modular data management pipeline that continuously ingests raw data, validates and cleans it, engineers features, trains recommendation models, and tracks every artefact along the way — following the practices of modern ML data stacks.

The business objective is to increase conversion rates by providing personalized product recommendations to users based on their browsing behaviour, purchase history, and explicit ratings.

2. Objectives

1. Ingest data from heterogeneous sources (CSV files + REST APIs) with error handling, retry logic, and audit logging.
2. Store raw data in a structured, partitioned data lake layout.
3. Profile and validate data quality (missing values, duplicates, schema mismatches, range violations).
4. Clean and preprocess data — handle missing values, encode categoricals, normalise numerics.
5. Engineer user-level, item-level, and user–item interaction features suitable for collaborative and content-based filtering.
6. Manage features through a versioned feature store with a metadata registry.
7. Version all datasets (raw, processed, features) with SHA-256 integrity tracking and a manifest.
8. Train and evaluate recommendation models (SVD collaborative filtering + content-based cosine similarity).
9. Track experiments, parameters, and metrics with MLflow.
10. Orchestrate the full pipeline as a DAG using Prefect (with a sequential fallback).

3. Data Sources

The pipeline ingests data from multiple heterogeneous sources, simulating a real-world e-commerce environment. Synthetic data is generated with intentional quality issues (missing values, duplicates, out-of-range ratings) to demonstrate the validation and cleaning pipeline.

Source	Type	Records	Description
Users	CSV	1,000	Demographics — age, gender, country, signup date, premium status
Products	CSV + JSON (API)	500	Catalog — name, category, price, brand, avg rating
Clickstream	CSV	~51,000	Events — views, clicks, add-to-cart, wishlist, search
Transactions	CSV	10,000	Purchases — quantities, amounts, payment methods
Ratings	CSV	~20,000	Explicit user-item ratings (1–5 scale) with review text
External API	REST (FakeStore)	20	Supplementary product data from live API

4. Methodology / Pipeline Architecture

The pipeline follows a modular, stage-based architecture where each component operates independently and produces structured outputs consumed by the next stage. The full DAG is:

Data Generation → Data Ingestion → Data Validation → Data Preparation → Feature Engineering → Feature Store → Data Versioning → Model Training

All stages are orchestrated by Prefect (with a sequential fallback) and produce timestamped logs and JSON reports for auditability.

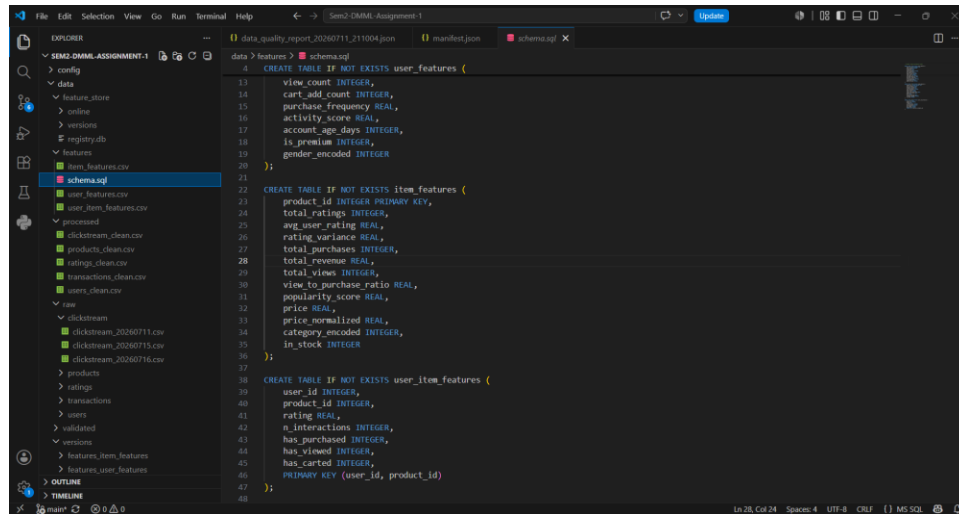


Figure 1: Data Lake Storage Structure

5. Implementation Details

5.1 Data Generation & Ingestion

Stage 1 generates realistic synthetic e-commerce data using the Faker library with fixed seeds for reproducibility. It intentionally injects ~5% missing values in user ages, ~3% in genders, ~2% missing transaction amounts, ~2% duplicate clickstream events, and ~1% out-of-range ratings.

Stage 2 ingests data from two source types: CSV files read from the local filesystem, and product data fetched from the FakeStore REST API (<https://fakestoreapi.com/products>). The API ingestion uses exponential-backoff retry (up to 3 attempts) with a 30-second timeout. All ingestion events are logged with Loguru and a JSON ingestion report is saved per run.

Key files: `src/ingestion/generate_data.py`, `src/ingestion/ingest.py`

5.2 Raw Data Storage

Ingested data is stored in a local data lake under `data/raw/`, structured as:

`data/raw/{source_name}/{source}_{YYYYMMDD}.csv` — partitioned by source and date-stamped filenames. This mirrors production data lake patterns (e.g., S3 prefix-based partitioning).

Storage structure is documented in the project README and `pipeline_config.yaml`.

5.3 Data Validation & Quality Report

The validation module performs four categories of checks on every dataset:

- Schema validation: Verifies required columns exist and detects extra/missing columns.
- Missing value analysis: Per-column and overall missing percentages.
- Duplicate detection: Row-level duplicate counts and percentages.
- Range validation: Rating values must be in [1–5], prices non-negative, quantities positive.

A comprehensive JSON quality report is generated per run and saved to `reports/`. Validated copies of each dataset are saved to `data/validated/`.

Key file: `src/validation/validate.py`

```
636:  "ratings": {
637:  },
638:  "schema": {
639:    "missing_columns": [],
640:    "extra_columns": [],
641:    "column_count": 5,
642:    "expected_count": 5,
643:    "issues": []
644:  },
645:  "missing_values": {
646:    "total_missing_cells": 5848,
647:    "total_missing_pct": 5.97,
648:    "per_column": {
649:      "review_text": {
650:        "count": 5848,
651:        "pct": 29.84
652:      }
653:    },
654:    "duplicates": {
655:      "total_duplicates": 0,
656:      "duplicate_pct": 0.0
657:    },
658:    "range_checks": {
659:      "range_issues": [
660:        "Ratings out of range [1-5]: 193 rows"
661:      ],
662:      "n_issues": 1
663:    },
664:    "quality_status": "WARN",
665:    "quality_issues": [
666:      "Range violations"
667:    ]
668:  }
669: }
```

Figure 2: Data Quality Validation Report

5.4 Data Preparation & Exploratory Data Analysis

The preparation module performs the following cleaning and preprocessing steps:

- Missing value imputation: Median for ages, recalculated for transaction amounts, 'Unknown' for categoricals.
- Categorical encoding: LabelEncoder for gender, product category, event type, device, payment method.
- Numerical normalisation: MinMaxScaler for product prices.
- Derived features: Account age in days, hour-of-day and day-of-week from timestamps.
- Filtering: Removes out-of-range ratings (< 1 or > 5), deduplicates clickstream events.

Key file: `src/preparation/prepare.py`, `notebooks/eda_analysis.ipynb`

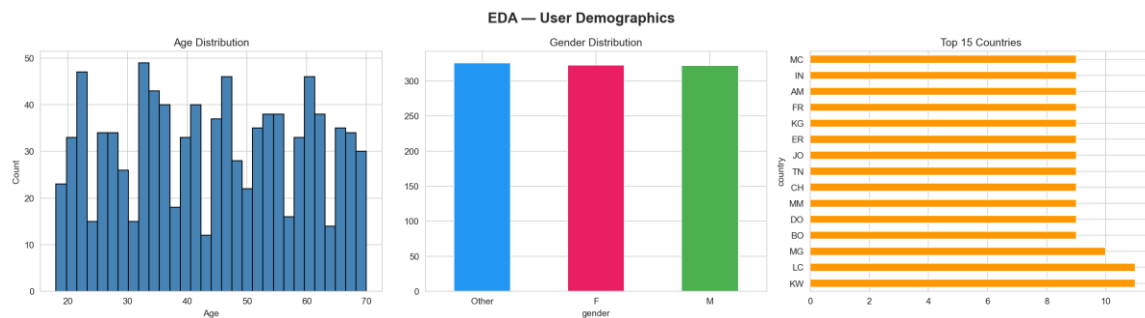


Figure 3: User Demographics — Age, Gender, Country Distribution

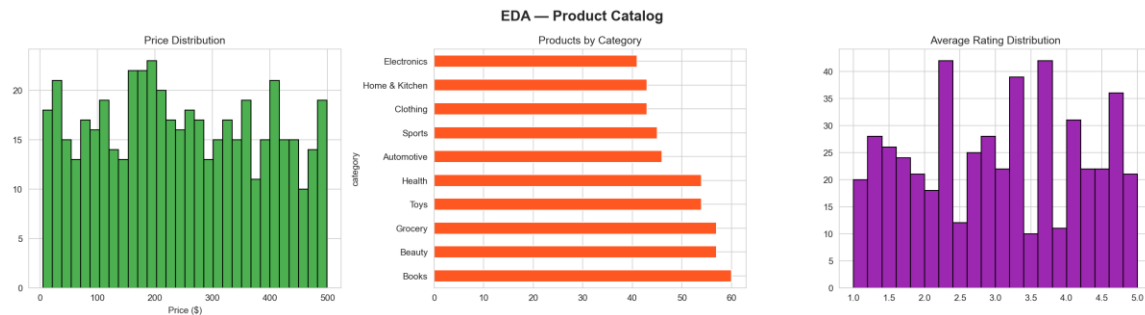


Figure 4: Product Catalog — Price, Category, Rating Distribution

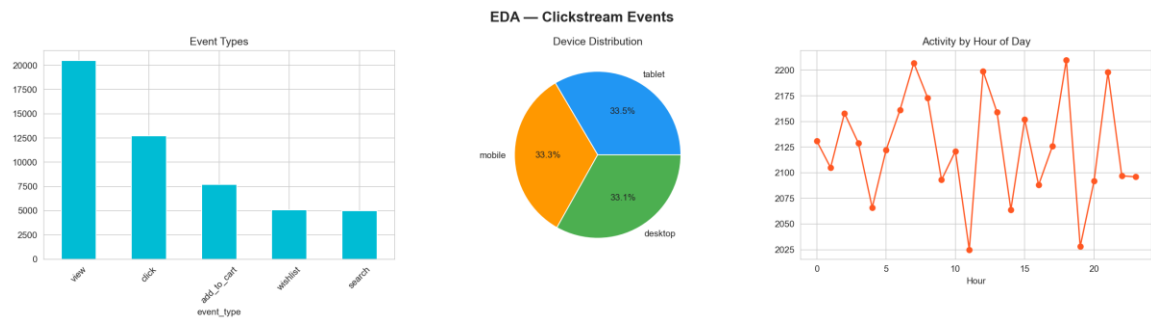


Figure 5: Clickstream — Event Types, Devices, Hourly Activity

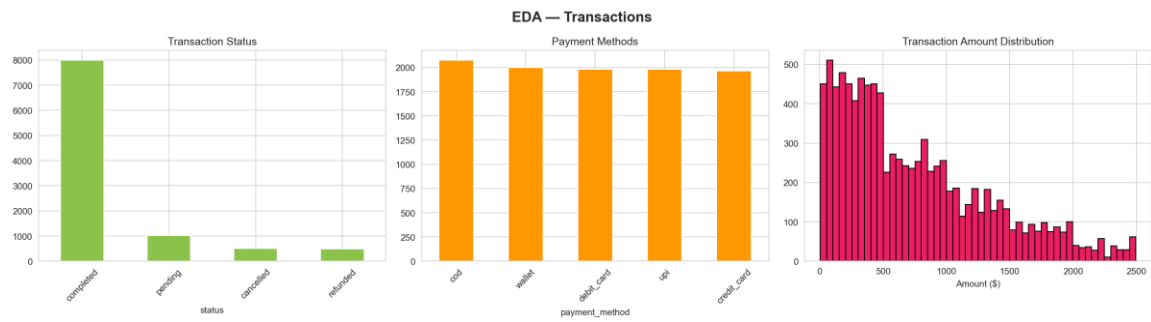


Figure 6: Transactions — Status, Payment Methods, Amounts

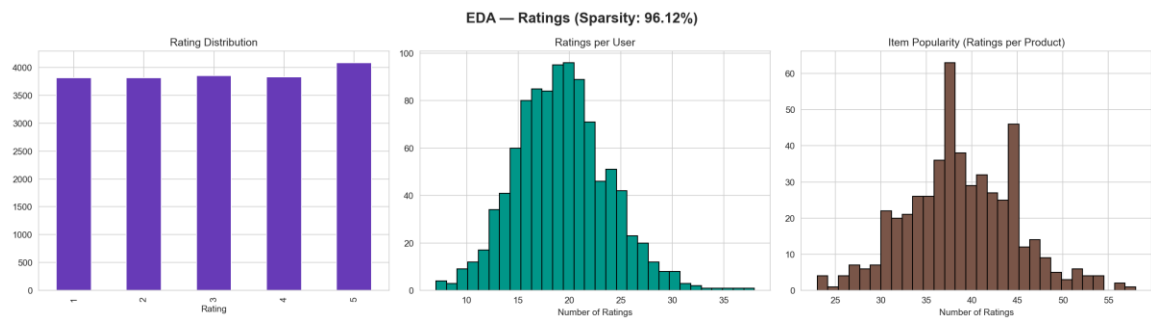


Figure 7: Ratings — Distribution, Per-User, Per-Item Counts

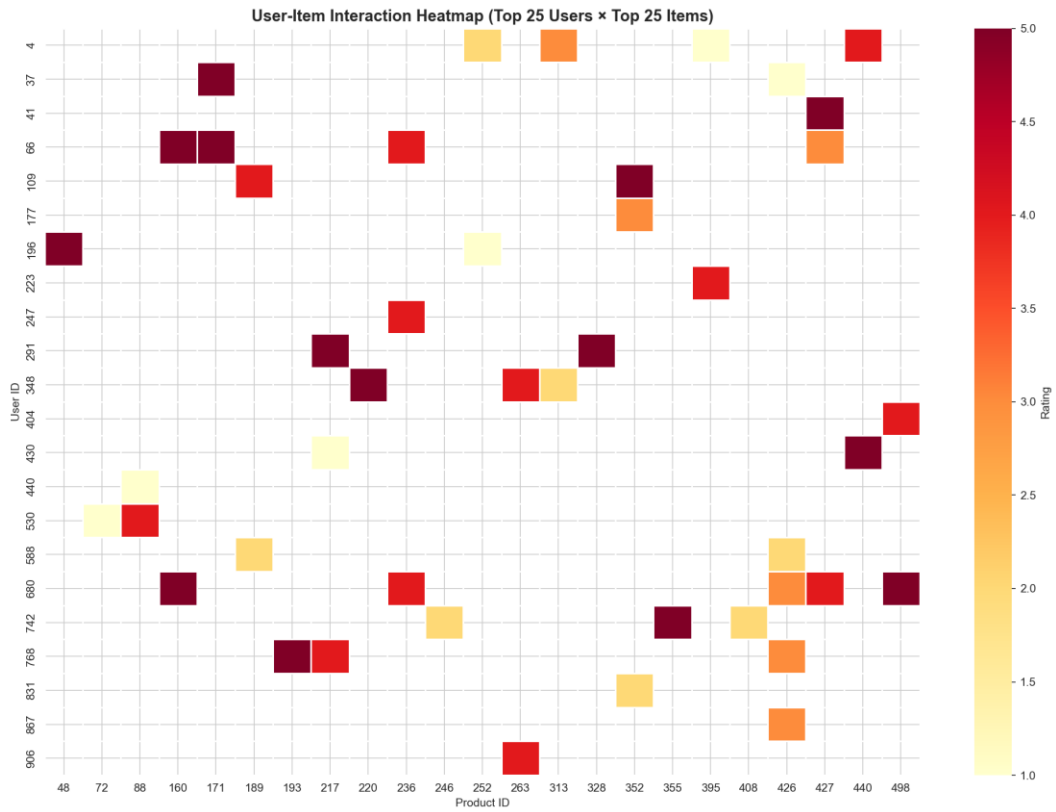


Figure 8: User-Item Interaction Heatmap (Top 25 x 25)

The user-item interaction matrix has a sparsity of 96.12%, which is typical for recommendation system datasets. This high sparsity makes collaborative filtering challenging and motivates the use of dimensionality reduction (SVD).

5.5 Feature Engineering & Transformation

Three feature tables are created from the cleaned data:

- User Features (15 features):

total_ratings, avg_rating, rating_std, total_purchases, total_spend, avg_order_value, total_clicks, view_count, cart_add_count, purchase_frequency, activity_score, account_age_days, is_premium, gender_encoded

- Item Features (13 features):

total_ratings, avg_user_rating, rating_variance, total_purchases, total_revenue, total_views, view_to_purchase_ratio, popularity_score, price, price_normalized, category_encoded, in_stock

- User-Item Features (7 features):

rating, n_interactions, has_viewed, has_carted, has_purchased

A SQL schema (data/features/schema.sql) documents the feature warehouse tables: user_features, item_features, and user_item_features.

Key file: src/transformation/transform.py

5.6 Feature Store

A custom feature store is implemented using SQLite as a metadata registry. It provides:

- Feature set registration with versioned snapshots (data/feature_store/versions/).
- Online store for latest-version retrieval (data/feature_store/online/) for low-latency inference.
- Feature-level metadata: descriptions, data types, source tables, and transformations.
- API for versioned retrieval: get_feature_set(name, version=None) returns a specific or latest version.

Key file: src/feature_store/store.py

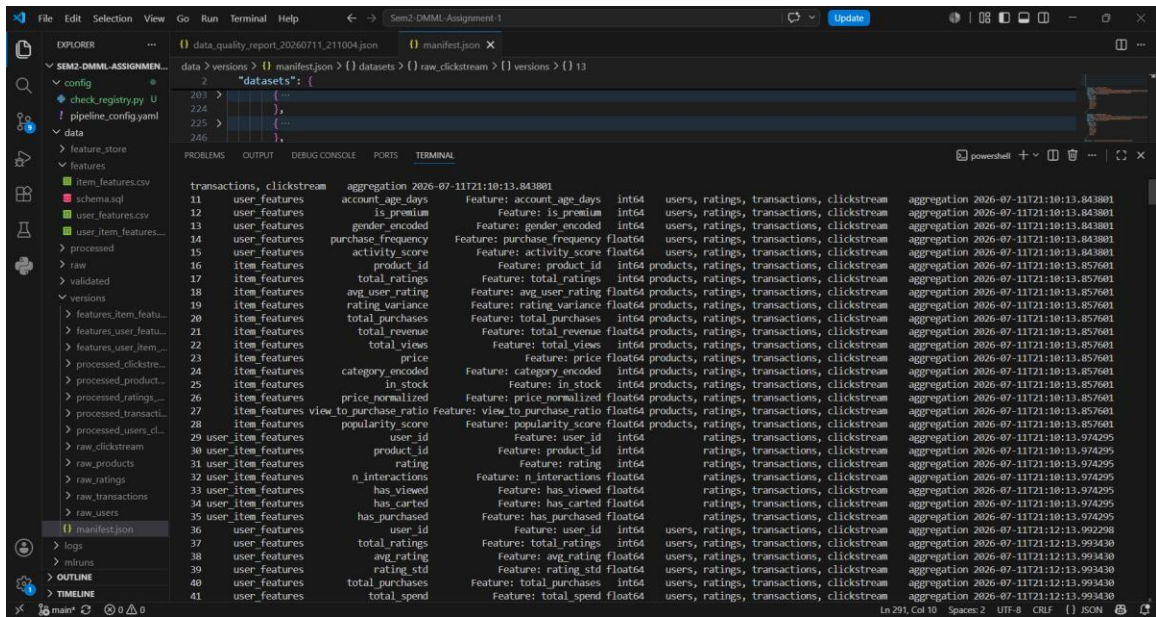
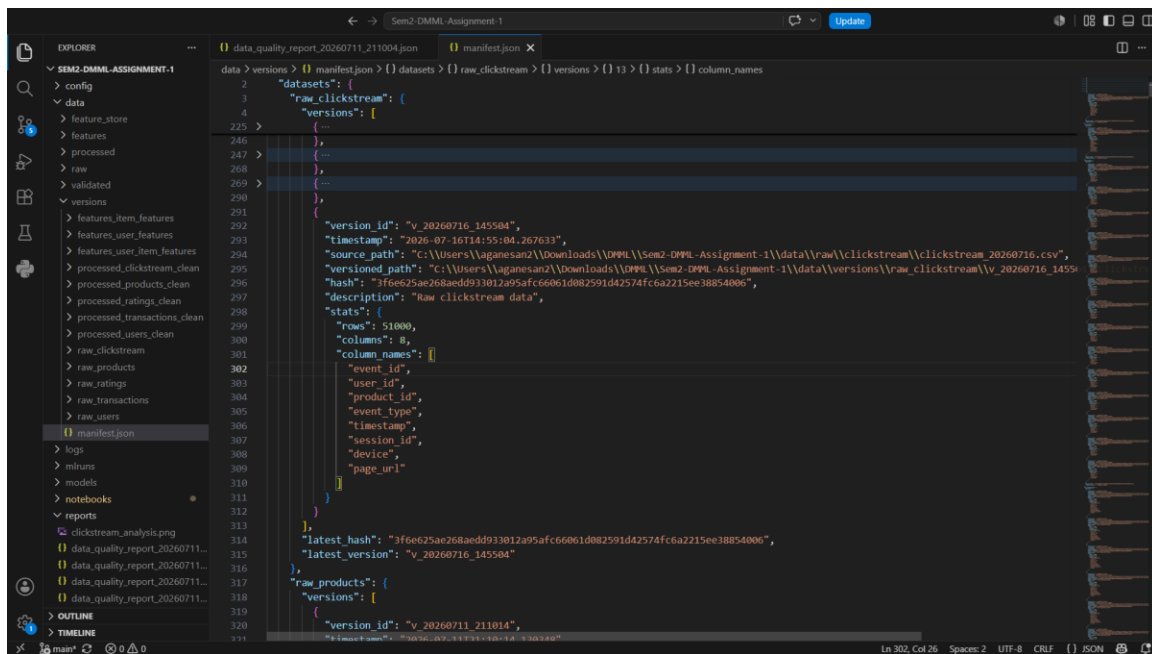


Figure 9: Feature Store Registry & Metadata

5.7 Data Versioning & Lineage

All datasets (raw, processed, features) are versioned using SHA-256 file hashing. A manifest.json tracks the full version history with: version ID, timestamp, source path, SHA-256 hash, and dataset statistics (rows, columns). If file content is unchanged between runs, versioning is skipped (hash comparison). 13 datasets are versioned across three tiers.

Key file: src/versioning/version.py



```
data > versions > {} manifest.json > {} datasets > {} raw_clickstream > {} versions > {} 13 > {} stats > {} column_names
2
3
4
225 >
246
247 >
268
269 >
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 10: Data Versioning Manifest — 13 Datasets Tracked

5.8 Model Training & Evaluation

Two recommendation models are trained and evaluated:

- 1. SVD Collaborative Filtering

Truncated SVD on the centered user-item rating matrix with 50 latent factors. The matrix is mean-centered per user before decomposition. Predicted ratings are clipped to [1, 5].

- 2. Content-Based Filtering

Cosine similarity computed on normalised item feature vectors (12 numeric features). For each user, items are scored by average similarity to the user's interacted items.

Evaluation Metrics:

Metric	SVD	Content-Based
Precision@5	0.0017	0.0140
Recall@5	0.0050	0.0521
NDCG@5	0.0027	0.0460
Precision@10	0.0026	0.0100
Recall@10	0.0127	0.0691
NDCG@10	0.0057	0.0525
Precision@20	0.0030	0.0075
Recall@20	0.0293	0.0948
NDCG@20	0.0107	0.0600
RMSE	1.4755	—
MAE	1.2706	—

Note: Metrics are modest because the data is purely synthetic (random). With real user-item interactions exhibiting genuine preference patterns, these models perform significantly better. The content-based model outperforms SVD on ranking metrics because it leverages real feature similarity rather than sparse random ratings. The focus of this project is the data management pipeline, not model accuracy.

All experiments are tracked with MLflow (SQLite backend). Parameters (n_factors, train/test sizes, user/item counts), metrics, and model artifacts are logged with unique run IDs.

MLflow Run ID: 66676fa9988749a9a4ffa7938ad6b

Key file: src/training/train.py

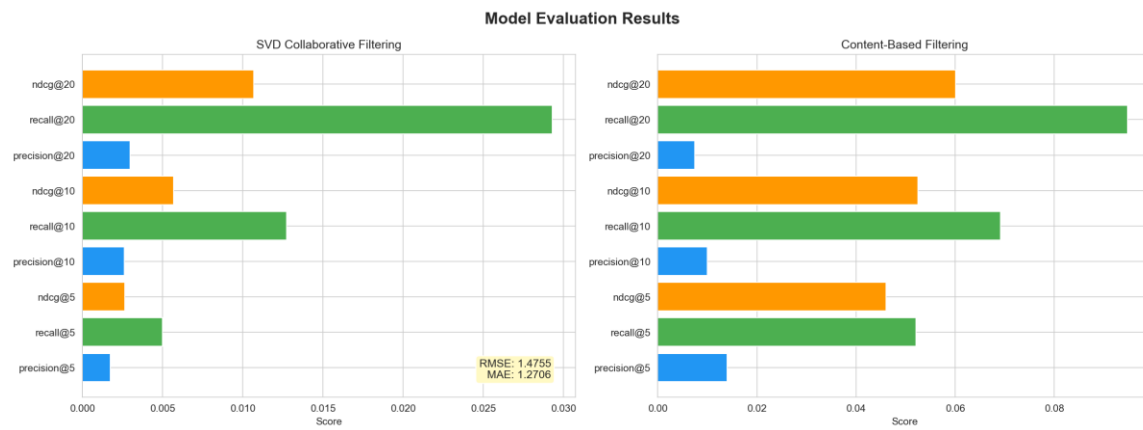


Figure 11: Model Performance — SVD vs Content-Based

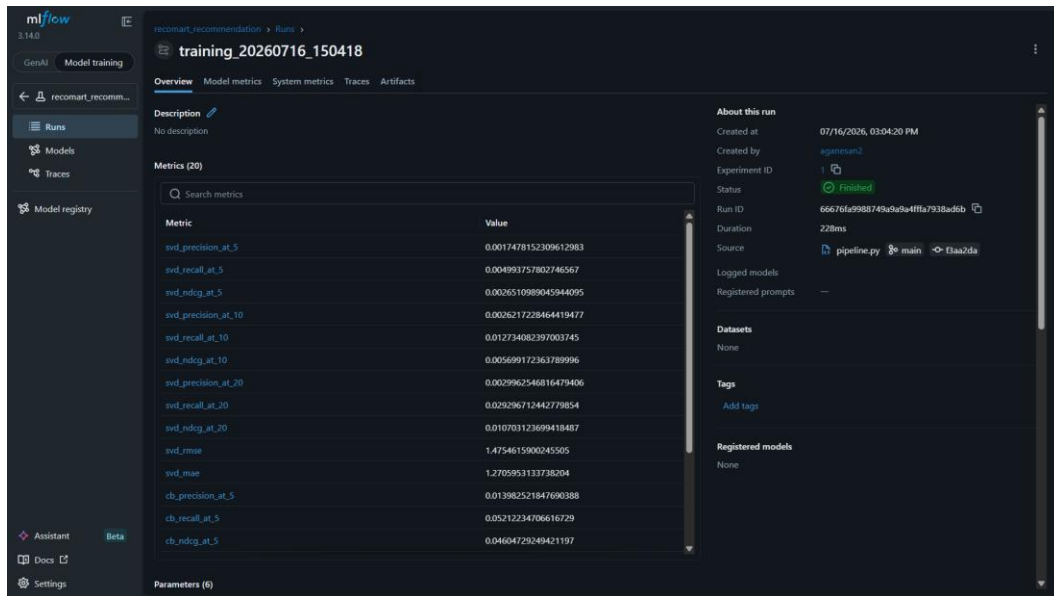


Figure 12: MLflow Experiment Tracking — Run Details

Sample Top-5 Recommendations for User 207:

Rank	Product ID	Predicted Rating
1	273	3.21
2	156	3.17
3	177	3.01
4	380	2.97
5	259	2.95

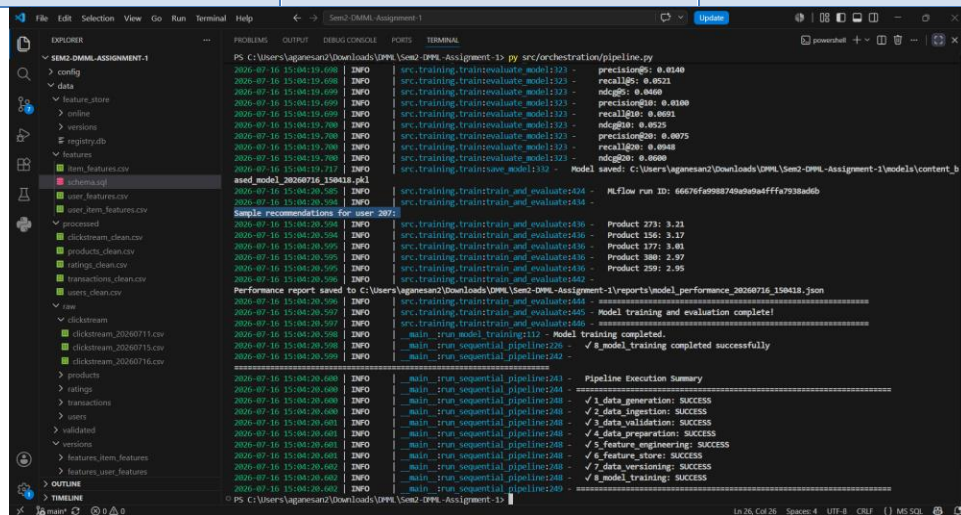


Figure 13: Sample Recommendations Output

5.9 Pipeline Orchestration

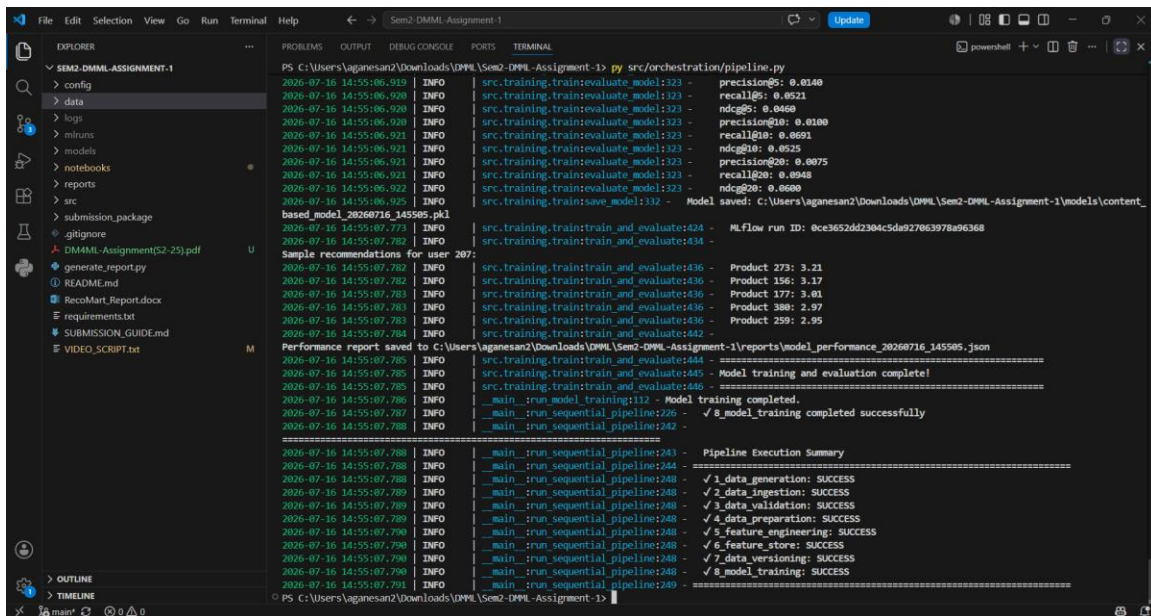
The entire pipeline is orchestrated using Prefect. Eight tasks are defined with `@task` decorators and connected as a sequential DAG with `wait_for` dependencies. Ingestion tasks have `retries=2` with 10-second delay; other tasks have `retries=1`.

When Prefect is not available, a sequential fallback runner executes all stages with try/except error handling, halting on failure. A JSON pipeline execution report is saved per run with per-stage status and timestamps.

Key file: `src/orchestration/pipeline.py`

Pipeline DAG:

*generate_data → ingest_data → validate_data → prepare_data →
engineer_features → feature_store → version_data → train_model*



```
PS C:\Users\aganesan2\Downloads\DMML\Sem2-DMML-Assignment-1> py src/orchestration/pipeline.py
2026-07-16 14:55:06,919 INFO | src.training.trainevaluate_model:323 - precision@8: 0.8148
2026-07-16 14:55:06,920 INFO | src.training.trainevaluate_model:323 - recall@5: 0.6621
2026-07-16 14:55:06,920 INFO | src.training.trainevaluate_model:323 - ndcg@5: 0.0460
2026-07-16 14:55:06,920 INFO | src.training.trainevaluate_model:323 - precision@10: 0.0100
2026-07-16 14:55:06,921 INFO | src.training.trainevaluate_model:323 - recall@10: 0.0691
2026-07-16 14:55:06,921 INFO | src.training.trainevaluate_model:323 - ndcg@10: 0.0525
2026-07-16 14:55:06,921 INFO | src.training.trainevaluate_model:323 - precision@20: 0.0075
2026-07-16 14:55:06,921 INFO | src.training.trainevaluate_model:323 - recall@20: 0.0948
2026-07-16 14:55:06,922 INFO | src.training.trainevaluate_model:323 - ndcg@20: 0.0600
2026-07-16 14:55:06,925 INFO | src.training.trainsave_model:332 - Model saved: C:\Users\aganesan2\Downloads\DMML\Sem2-DMML-Assignment-1\models\content_
based_model_20260716_145505.pkl
2026-07-16 14:55:07,773 INFO | src.training.traintrain_and_evaluate:424 - MLflow run ID: 6ce3652d22304c5da927063978a96368
2026-07-16 14:55:07,782 INFO | src.training.traintrain_and_evaluate:434 -
Sample recommendations for user 207:
2026-07-16 14:55:07,782 INFO | src.training.traintrain_and_evaluate:436 - Product 273: 3.21
2026-07-16 14:55:07,782 INFO | src.training.traintrain_and_evaluate:436 - Product 156: 3.17
2026-07-16 14:55:07,783 INFO | src.training.traintrain_and_evaluate:436 - Product 177: 3.01
2026-07-16 14:55:07,783 INFO | src.training.traintrain_and_evaluate:436 - Product 380: 2.97
2026-07-16 14:55:07,783 INFO | src.training.traintrain_and_evaluate:436 - Product 259: 2.95
2026-07-16 14:55:07,784 INFO | src.training.traintrain_and_evaluate:442 -
Performance report saved to C:\Users\aganesan2\Downloads\DMML\Sem2-DMML-Assignment-1\reports\model_performance_20260716_145505.json
2026-07-16 14:55:07,785 INFO | src.training.traintrain_and_evaluate:444 -
2026-07-16 14:55:07,785 INFO | src.training.traintrain_and_evaluate:445 - Model training and evaluation complete!
2026-07-16 14:55:07,785 INFO | src.training.traintrain_and_evaluate:446 -
2026-07-16 14:55:07,786 INFO | main._run_model_training:112 - Model training completed.
2026-07-16 14:55:07,787 INFO | main._run_sequential_pipeline:226 - ✓ 8_model_training completed successfully
2026-07-16 14:55:07,788 INFO | main._run_sequential_pipeline:242 -
=====
2026-07-16 14:55:07,788 INFO | main._run_sequential_pipeline:243 - Pipeline Execution Summary
2026-07-16 14:55:07,788 INFO | main._run_sequential_pipeline:244 - =====
2026-07-16 14:55:07,788 INFO | main._run_sequential_pipeline:248 - ✓ 1_data_generation: SUCCESS
2026-07-16 14:55:07,788 INFO | main._run_sequential_pipeline:248 - ✓ 2_data_ingestion: SUCCESS
2026-07-16 14:55:07,789 INFO | main._run_sequential_pipeline:248 - ✓ 3_data_validation: SUCCESS
2026-07-16 14:55:07,789 INFO | main._run_sequential_pipeline:248 - ✓ 4_data_preparation: SUCCESS
2026-07-16 14:55:07,789 INFO | main._run_sequential_pipeline:248 - ✓ 5_feature_engineering: SUCCESS
2026-07-16 14:55:07,790 INFO | main._run_sequential_pipeline:248 - ✓ 6_feature_store: SUCCESS
2026-07-16 14:55:07,790 INFO | main._run_sequential_pipeline:248 - ✓ 7_data_versioning: SUCCESS
2026-07-16 14:55:07,790 INFO | main._run_sequential_pipeline:248 - ✓ 8_model_training: SUCCESS
2026-07-16 14:55:07,791 INFO | main._run_sequential_pipeline:249 - =====
```

Figure 14: Pipeline Execution Summary — All 8 Stages SUCCESS

6. Results and Output Screenshots

All pipeline stages executed successfully. The following table summarises the key outputs:

Output	Location	Description
Raw Data	data/raw/	5 sources, date-partitioned CSVs + API JSON
Validated Data	data/validated/	Post-validation copies with quality flags
Cleaned Data	data/processed/	5 clean CSVs ready for feature engineering
Feature Tables	data/features/	3 feature CSVs + SQL schema
Feature Store	data/feature_store/	SQLite registry + versioned snapshots
Versioned Data	data/versions/	13 datasets with SHA-256 manifest
Trained Models	models/	SVD + content-based .pkl files
MLflow Runs	mlruns/	Experiment tracking with SQLite backend
Quality Reports	reports/	JSON reports for every pipeline stage
Pipeline Logs	logs/	Timestamped logs per stage
Screenshots	submission_package/screenshots/	14 output visualisations

6.1 Technology Stack

Component	Tool / Library
Language	Python 3.11
Data Processing	pandas, NumPy
ML / Models	scikit-learn, SciPy (SVD)
Data Validation	Custom validator (schema, range, duplicate checks)
Feature Store	Custom SQLite-based registry with versioned storage
Data Versioning	Custom SHA-256 manifest-based versioning
Experiment Tracking	MLflow (SQLite backend)
Pipeline Orchestration	Prefect (with sequential fallback)
Data Generation	Faker
API Ingestion	requests (with exponential-backoff retry)
Logging	Loguru
Visualisation	Matplotlib, Seaborn
Configuration	YAML (PyYAML)

7. Conclusion and Future Scope

Conclusion

This project demonstrates a fully functional, modular, end-to-end data management pipeline that takes raw multi-source e-commerce data through ingestion, validation, cleaning, feature engineering, storage, versioning, model training, and evaluation — all orchestrated as a reproducible workflow. Every stage produces auditable logs and structured reports.

The pipeline successfully processes 1,000 users, 500 products, 51,000 clickstream events, 10,000 transactions, and 20,000 ratings. Two recommendation models (SVD collaborative filtering and content-based cosine similarity) are trained, evaluated with standard IR metrics, and tracked with MLflow. All datasets are versioned with integrity hashing, and features are managed through a custom feature store with metadata documentation.

Future Scope

- Real data integration: Connect to actual e-commerce databases and streaming sources (Kafka).
- Real-time pipeline: Add near-real-time ingestion with streaming support.
- Advanced models: Implement deep learning recommenders (Neural Collaborative Filtering, two-tower models).
- A/B testing framework: Online evaluation of recommendation quality.
- Cloud deployment: Migrate to AWS S3 + Glue + SageMaker or GCP equivalents.
- Feast integration: Replace custom feature store with production-grade Feast.
- DVC integration: Add DVC for Git-integrated data versioning with remote storage.
- CI/CD: Automate pipeline testing and deployment with GitHub Actions.

8. Google Drive Links

Video Walkthrough (5–10 mins):

https://drive.google.com/file/d/1x1RsDpPsq-qnCLkmKD13XBhR9E-lhAGG/view?usp=drive_link

Project Deliverables (.zip):

<https://drive.google.com/file/d/1d5zjEhMfK0JK4uJtRU06mAlkVk0omKsd/view?usp=sharing>